SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – High (Coding Challenge)

Course Code:	CSA0501	**Course Title:**	**Programming in Java**
CO Assessed:	CO1 – Precision (BL3,BL4)	**Assessment:**	Assessment Tool 1 – Coding Challenge
Weightage:	40% of CIA	**Total Marks:**	100 (5 Questions × 20 Marks)
CO1 Statement:	*Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*		
Student Name:	M Saravanan	**Reg. No.:**	1921260140
Section / Batch:	2026	**Date:**	22.07.2026

Code of Conduct

I M Saravanan *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: M Saravanan

Instructions

- *This test contains 10 questions, each carrying 10 marks. Total: 100 Marks.*
- *No negative marking. Unanswered questions carry zero marks.*
- *No calculators or electronic devices permitted.*
- *Duration: 90 minutes.*

Rubrics for Calculation

Criteria	Weigh tage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks	
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints		
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach		
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results		
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints		
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases		

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____

ANNEXURE

Coding Challenge

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

4. Bank Account System using Data Hiding

Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

- Undergraduate students,
- Postgraduate students,
- Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

7. Hospital Management System using Interfaces

Develop a hospital management system with an interface MedicalRecord containing methods addRecord() and displayRecord(). Implement the interface in classes Patient and Doctor. Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

8. Online Shopping Order System using Packages

Create two packages:

- shopping.product
- shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

9. Student Result Analyzer using Arrays of Objects

Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

Date: _____	Date: _____	Date: _____

Experiment 1: Library Book Management using Classes & Encapsulation

Aim

To develop a Library Book Management System using Java classes and encapsulation by implementing constructors, getters, setters, and methods to add, issue, return, search, and display books using an array of objects.

Algorithm

1. Start the program.
2. Create a Bookclass with private data members:
   - bookId
   - title
   - author
   - price
   - issued
3. Create constructors, getters, setters, and methods:
   - issueBook()
   - returnBook()
   - display()
4. Create an array of Book objects.
5. Display the menu:
   - Add Book
   - Display Books
   - Search Book
   - Issue Book
   - Return Book
   - Exit
6. Read the user's choice.
7. Perform the selected operation.
8. Repeat the menu until the user selects Exit.
9. Stop the program.

Java Code

import java.util.Scanner;

```
class Book {
    private int bookId;
    private String title;
    private String author;
    private double price;
    private boolean issued;

    Book(int id, String t, String a, double p) {
        bookId = id;
        title = t;
        author = a;
        price = p;
        issued = false;
    }

    public int getBookId() {
        return bookId;
    }

    public String getTitle() {
        return title;
    }
```

```
    public void issueBook() {
        if (!issued) {
            issued = true;
            System.out.println("Book Issued Successfully");
        } else {
            System.out.println("Book Already Issued");
        }
    }

    public void returnBook() {
        if (issued) {
            issued = false;
            System.out.println("Book Returned Successfully");
        } else {
            System.out.println("Book Was Not Issued");
        }
    }

    public void display() {
        System.out.println(bookId + " " + title + " " + author + " " + price + " "
+ (issued ? "Issued" : "Available"));
    }
}
```

```
public class LibraryManagement {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Book[] books = new Book[10];
        int count = 0, choice;

        do {
            System.out.println("\n===== Library Menu =====");
            System.out.println("1. Add Book");
            System.out.println("2. Display Books");
            System.out.println("3. Search Book");
            System.out.println("4. Issue Book");
            System.out.println("5. Return Book");
            System.out.println("6. Exit");

            System.out.print("Enter Choice: ");
            choice = sc.nextInt();

            switch (choice) {

                case 1:
```

```
    System.out.print("Book ID: ");
    int id = sc.nextInt();
    sc.nextLine();

    System.out.print("Title: ");
    String title = sc.nextLine();

    System.out.print("Author: ");
    String author = sc.nextLine();

    System.out.print("Price: ");
    double price = sc.nextDouble();

    books[count++] = new Book(id, title, author, price);
    System.out.println("Book Added Successfully");
    break;

case 2:
    for (int i = 0; i < count; i++)
        books[i].display();
    break;

case 3:
```

```
    System.out.print("Enter Title: ");
    sc.nextLine();
    String search = sc.nextLine();

    boolean found = false;
    for (int i = 0; i < count; i++) {
        if (books[i].getTitle().equalsIgnoreCase(search)) {
            books[i].display();
            found = true;
        }
    }

    if (!found)
        System.out.println("Book Not Found");
    break;

case 4:
    System.out.print("Enter Book ID: ");
    id = sc.nextInt();

    for (int i = 0; i < count; i++)
        if (books[i].getBookId() == id)
            books[i].issueBook();
```

```
            break;

        case 5:
            System.out.print("Enter Book ID: ");
            id = sc.nextInt();

            for (int i = 0; i < count; i++)
                if (books[i].getBookId() == id)
                    books[i].returnBook();
            break;

        case 6:
            System.out.println("Thank You");
            break;

        default:
            System.out.println("Invalid Choice");
      }

   } while (choice != 6);

   sc.close();
}
```

}

Sample Input

1

101

Java Programming

James Gosling

650

Sample Output

===== Library Menu =====

1. Add Book
2. Display Books
3. Search Book
4. Issue Book
5. Return Book
6. Exit

Enter Choice: 1

Book ID: 101

Title: Java Programming

Author: James Gosling

Price: 650

Book Added Successfully

Experiment 2: Employee Payroll System using Inheritance

Aim

To develop an Employee Payroll System using Java Inheritance and Runtime Polymorphism by creating a base class Employee and derived classes PermanentEmployee and ContractEmployee to calculate and display employee salaries.

Algorithm

1. Start the program.
2. Create a base class Employeewith data members:
   - empId
   - name
3. Create a constructor to initialize employee details.
4. Create a method calculateSalary().
5. Create two derived classes:
   - PermanentEmployee
   - ContractEmployee
6. Override calculateSalary() in each subclass.
7. Accept employee details from the user.
8. Use runtime polymorphism to calculate and display salary.
9. Stop the program.

Java Code

```
import java.util.Scanner;


class Employee {

   int empId;

   String name;


   Employee(int id, String n) {
```

```
        empId = id;
        name = n;
    }

    void calculateSalary() {
        System.out.println("Salary Calculation");
    }

    void display() {
        System.out.println("Employee ID : " + empId);
        System.out.println("Name : " + name);
    }
}

class PermanentEmployee extends Employee {
    double basic, hra;

    PermanentEmployee(int id, String n, double b, double h) {
        super(id, n);
        basic = b;
        hra = h;
    }
```

```
    void calculateSalary() {
        System.out.println("Salary : " + (basic + hra));
    }
}

class ContractEmployee extends Employee {
    int hours;
    double rate;

    ContractEmployee(int id, String n, int h, double r) {
        super(id, n);
        hours = h;
        rate = r;
    }

    void calculateSalary() {
        System.out.println("Salary : " + (hours * rate));
    }
}

public class PayrollSystem {
    public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);

System.out.println("1. Permanent Employee");
System.out.println("2. Contract Employee");
System.out.print("Enter Choice: ");
int ch = sc.nextInt();

Employee emp;

if (ch == 1) {

    System.out.print("Employee ID: ");
    int id = sc.nextInt();
    sc.nextLine();

    System.out.print("Name: ");
    String name = sc.nextLine();

    System.out.print("Basic Salary: ");
    double basic = sc.nextDouble();

    System.out.print("HRA: ");
    double hra = sc.nextDouble();
```

```
    emp = new PermanentEmployee(id, name, basic, hra);

} else {

    System.out.print("Employee ID: ");
    int id = sc.nextInt();
    sc.nextLine();

    System.out.print("Name: ");
    String name = sc.nextLine();

    System.out.print("Hours Worked: ");
    int hours = sc.nextInt();

    System.out.print("Rate Per Hour: ");
    double rate = sc.nextDouble();

    emp = new ContractEmployee(id, name, hours, rate);
}

emp.display();
emp.calculateSalary();
```

```
        sc.close();
    }
}
```

Sample Input

```
1
101
Rahul
30000
5000
```

Sample Output

```
1. Permanent Employee
2. Contract Employee
Enter Choice: 2

Employee ID: 102
Name: Priya
Hours Worked: 25
Rate Per Hour: 600

Employee ID : 102
Name : Priya
```

Salary : 15000.0

Experiment 3: Vehicle Rental System using Runtime Polymorphism

Aim

To develop a Vehicle Rental System using Java Runtime Polymorphism by creating a superclass Vehicle and subclasses Car, Bike, and Bus to calculate rental charges dynamically.

Algorithm

1. Start the program.
2. Create a superclass Vehiclewith data members:
   - vehicleNo
   - vehicleName
3. Create a constructor to initialize vehicle details.
4. Create a method calculateRent(int days).
5. Create subclasses:
   - Car
   - Bike
   - Bus
6. Override calculateRent(int days) in each subclass.
7. Store vehicle objects in an array of Vehicle references.
8. Accept the number of rental days.
9. Display vehicle details and calculate rent using runtime polymorphism.
10. Stop the program.

Java Code

```
import java.util.Scanner;


class Vehicle {

   int vehicleNo;

   String vehicleName;


   Vehicle(int no, String name) {
```

```
        vehicleNo = no;
        vehicleName = name;
    }

    void calculateRent(int days) {
        System.out.println("Rent Calculation");
    }

    void display() {
        System.out.println("Vehicle No : " + vehicleNo);
        System.out.println("Vehicle Name : " + vehicleName);
    }
}

class Car extends Vehicle {
    Car(int no, String name) {
        super(no, name);
    }

    void calculateRent(int days) {
        System.out.println("Car Rent : " + (days * 1000));
    }
}
```

```
class Bike extends Vehicle {
    Bike(int no, String name) {
        super(no, name);
    }

    void calculateRent(int days) {
        System.out.println("Bike Rent : " + (days * 500));
    }
}

class Bus extends Vehicle {
    Bus(int no, String name) {
        super(no, name);
    }

    void calculateRent(int days) {
        System.out.println("Bus Rent : " + (days * 2000));
    }
}

public class VehicleRental {
    public static void main(String[] args) {
```

```
    Scanner sc = new Scanner(System.in);

    Vehicle[] v = new Vehicle[3];

    v[0] = new Car(101, "Swift");
    v[1] = new Bike(102, "Apache");
    v[2] = new Bus(103, "Volvo");

    System.out.println("1. Car");
    System.out.println("2. Bike");
    System.out.println("3. Bus");
    System.out.print("Enter Choice: ");
    int ch = sc.nextInt();

    System.out.print("Enter Rental Days: ");
    int days = sc.nextInt();

    v[ch - 1].display();
    v[ch - 1].calculateRent(days);

    sc.close();
}
```

}

Sample Input

1

5

Sample Output

1. Car
2. Bike
3. Bus
4. Enter Choice: 1
5. Enter Rental Days: 5

Vehicle No : 101

Vehicle Name : Swift

Car Rent : 5000

Experiment 4: Bank Account System using Data Hiding

Aim

To develop a **Bank Account System** using Java **Data Hiding (Encapsulation)** by creating a BankAccount class with private data members and implementing deposit, withdrawal, and balance inquiry with proper validation.

Algorithm

1. Start the program.
2. Create a BankAccountclass with private data members:
   - accountNumber
   - holderName
   - balance
3. Create a constructor to initialize account details.
4. Implement methods:
   - deposit()
   - withdraw()
   - showBalance()
5. Validate deposit and withdrawal amounts.
6. Display a menu:
   - Deposit
   - Withdraw
   - Balance Inquiry
   - Exit
7. Read the user's choice.
8. Perform the selected operation.
9. Repeat until the user selects Exit.
10. Stop the program.

Java Code

```
import java.util.Scanner;


class BankAccount {
    private int accountNumber;
    private String holderName;
```

```
private double balance;

BankAccount(int accNo, String name, double bal) {
    accountNumber = accNo;
    holderName = name;
    balance = bal;
}

public void deposit(double amount) {
    if (amount > 0) {
        balance += amount;
        System.out.println("Amount Deposited Successfully");
    } else {
        System.out.println("Invalid Deposit Amount");
    }
}

public void withdraw(double amount) {
    if (amount > 0 && amount <= balance) {
        balance -= amount;
        System.out.println("Amount Withdrawn Successfully");
    } else {
        System.out.println("Insufficient Balance or Invalid Amount");
```

```
        }
    }

    public void showBalance() {
        System.out.println("Account Number : " + accountNumber);
        System.out.println("Holder Name : " + holderName);
        System.out.println("Balance : " + balance);
    }
}

public class BankSystem {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        BankAccount acc = new BankAccount(101, "Rahul", 5000);

        int choice;

        do {
            System.out.println("\n===== Bank Menu =====");
            System.out.println("1. Deposit");
            System.out.println("2. Withdraw");
```

```
System.out.println("3. Balance Inquiry");
System.out.println("4. Exit");

System.out.print("Enter Choice: ");
choice = sc.nextInt();

switch (choice) {

   case 1:
      System.out.print("Enter Deposit Amount: ");
      double d = sc.nextDouble();
      acc.deposit(d);
      break;

   case 2:
      System.out.print("Enter Withdrawal Amount: ");
      double w = sc.nextDouble();
      acc.withdraw(w);
      break;

   case 3:
      acc.showBalance();
      break;
```

```
            case 4:
                System.out.println("Thank You");
                break;

            default:
                System.out.println("Invalid Choice");
        }

    } while (choice != 4);

    sc.close();
  }
}
```

Sample Input

```
1
2000
3
2
1000
3
4
```

Sample Output

===== Bank Menu =====

1. **Deposit**
2. **Withdraw**
3. **Balance Inquiry**
4. **Exit**

Enter Choice: 1

Enter Deposit Amount: 2000

Amount Deposited Successfully

Enter Choice: 3

Account Number : 101

Holder Name : Rahul

Balance : 7000.0

Enter Choice: 2

Enter Withdrawal Amount: 1000

Amount Withdrawn Successfully

Enter Choice: 3

Account Number : 101

Holder Name : Rahul

Balance : 6000.0

Enter Choice: 4

Thank You

Experiment 5: University Admission System using Method Overloading

Aim

To develop a **University Admission System** using Java **Method Overloading** by implementing overloaded calculateFee() methods for Undergraduate, Postgraduate, and Scholarship students.

Algorithm

1. Start the program.
2. Create a class Admission.
3. Create overloaded methods:
   - calculateFee(double fee) for Undergraduate students.
   - calculateFee(double fee, double labFee) for Postgraduate students.
   - calculateFee(double fee, double scholarship, boolean status) for Scholarship students.
4. Display a menu:
   - Undergraduate
   - Postgraduate
   - Scholarship
   - Exit
5. Read the user's choice.
6. Call the appropriate overloaded method.
7. Display the calculated fee.
8. Repeat until the user chooses Exit.
9. Stop the program.

Java Code

```
import java.util.Scanner;


class Admission {


   void calculateFee(double fee) {

      System.out.println("Undergraduate Fee : " + fee);
```

```
    }

    void calculateFee(double fee, double labFee) {
        System.out.println("Postgraduate Fee : " + (fee + labFee));
    }

    void calculateFee(double fee, double scholarship, boolean status) {
        if (status)
            System.out.println("Scholarship Fee : " + (fee - scholarship));
        else
            System.out.println("Scholarship Not Available");
    }
}

public class UniversityAdmission {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        Admission a = new Admission();

        int choice;

        do {
```

```
System.out.println("\n===== Admission Menu =====");
System.out.println("1. Undergraduate");
System.out.println("2. Postgraduate");
System.out.println("3. Scholarship");
System.out.println("4. Exit");

System.out.print("Enter Choice: ");
choice = sc.nextInt();

switch (choice) {

   case 1:
      System.out.print("Enter UG Fee: ");
      double ug = sc.nextDouble();
      a.calculateFee(ug);
      break;

   case 2:
      System.out.print("Enter PG Fee: ");
      double pg = sc.nextDouble();

      System.out.print("Enter Lab Fee: ");
      double lab = sc.nextDouble();
```

```
            a.calculateFee(pg, lab);
            break;

        case 3:
            System.out.print("Enter Course Fee: ");
            double fee = sc.nextDouble();

            System.out.print("Enter Scholarship Amount: ");
            double scholarship = sc.nextDouble();

            a.calculateFee(fee, scholarship, true);
            break;

        case 4:
            System.out.println("Thank You");
            break;

        default:
            System.out.println("Invalid Choice");
    }

} while (choice != 4);
```

```
        sc.close();
    }
}
```

Sample Input

```
1
50000
2
60000
5000
3
50000
10000
4
```

Sample Output

```
===== Admission Menu =====
1. Undergraduate
2. Postgraduate
3. Scholarship
4. Exit
Enter Choice: 1
Enter UG Fee: 50000
Undergraduate Fee : 50000.0
```

Enter Choice: 2

Enter PG Fee: 60000

Enter Lab Fee: 5000

Postgraduate Fee : 65000.0

Enter Choice: 3

Enter Course Fee: 50000

Enter Scholarship Amount: 10000

Scholarship Fee : 40000.0

Enter Choice: 4

Thank You

Experiment 6: Shape Area Calculator using Abstract Class & Polymorphism

Aim

To develop a **Shape Area Calculator** using Java **Abstract Class** and **Runtime Polymorphism** by creating an abstract class Shape and implementing subclasses Circle, Rectangle, and Triangle to calculate areas dynamically.

Algorithm

1. Start the program.
2. Create an abstract class Shape with an abstract method area().
3. Create subclasses Circle, Rectangle, and Triangle.
4. Override the area() method in each subclass.
5. Store the objects in an array of Shape references.
6. Call the area() method using runtime polymorphism.
7. Display the area of each shape.
8. Stop the program.

Java Code

```
abstract class Shape {

   abstract void area();

}


class Circle extends Shape {

   double r;


   Circle(double r) {

      this.r = r;

   }
```

```
    void area() {
        System.out.println("Circle Area = " + (3.14 * r * r));
    }
}

class Rectangle extends Shape {
    double l, b;

    Rectangle(double l, double b) {
        this.l = l;
        this.b = b;
    }

    void area() {
        System.out.println("Rectangle Area = " + (l * b));
    }
}

class Triangle extends Shape {
    double base, height;

    Triangle(double base, double height) {
```

```
        this.base = base;
        this.height = height;
    }

    void area() {
        System.out.println("Triangle Area = " + (0.5 * base * height));
    }
}

public class ShapeAreaCalculator {
    public static void main(String[] args) {

        Shape[] s = new Shape[3];

        s[0] = new Circle(7);
        s[1] = new Rectangle(10, 5);
        s[2] = new Triangle(8, 6);

        for (Shape shape : s) {
            shape.area();
        }
    }
}
```

Sample Input

Circle Radius = 7

Rectangle Length = 10

Rectangle Breadth = 5

Triangle Base = 8

Triangle Height = 6

Sample Output

Circle Area = 153.86

Rectangle Area = 50.0

Triangle Area = 24.0

Experiment 7: Hospital Management System using Interfaces

Aim

To develop a **Hospital Management System** using Java **Interfaces** by creating an interface MedicalRecord with methods addRecord() and displayRecord(), and implementing it in the classes Patient and Doctor.

Algorithm

1. Start the program.
2. Create an interface MedicalRecordwith methods:
   - addRecord()
   - displayRecord()
3. Create a class Patient that implements the interface.
4. Create a class Doctor that implements the interface.
5. Accept patient and doctor details.
6. Store the details using addRecord().
7. Display the details using displayRecord().
8. Stop the program.

Java Code

```
import java.util.Scanner;


interface MedicalRecord {
   void addRecord();
   void displayRecord();
}


class Patient implements MedicalRecord {
   int patientId;
   String patientName;
```

```
    Scanner sc = new Scanner(System.in);

    public void addRecord() {
        System.out.print("Enter Patient ID: ");
        patientId = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Patient Name: ");
        patientName = sc.nextLine();
    }

    public void displayRecord() {
        System.out.println("\nPatient Record");
        System.out.println("Patient ID : " + patientId);
        System.out.println("Patient Name : " + patientName);
    }
}

class Doctor implements MedicalRecord {
    int doctorId;
    String doctorName;
    Scanner sc = new Scanner(System.in);
```

```
    public void addRecord() {
        System.out.print("Enter Doctor ID: ");
        doctorId = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Doctor Name: ");
        doctorName = sc.nextLine();
    }

    public void displayRecord() {
        System.out.println("\nDoctor Record");
        System.out.println("Doctor ID : " + doctorId);
        System.out.println("Doctor Name : " + doctorName);
    }
}

public class HospitalManagement {
    public static void main(String[] args) {

        MedicalRecord p = new Patient();
        MedicalRecord d = new Doctor();

        p.addRecord();
```

```
        d.addRecord();


        p.displayRecord();
        d.displayRecord();
    }
}
```

Sample Input

```
101
Rahul
201
Dr. Kumar
```

Sample Output

```
Enter Patient ID: 101
Enter Patient Name: Rahul

Enter Doctor ID: 201
Enter Doctor Name: Dr. Kumar

Patient Record
Patient ID : 101
Patient Name : Rahul
```

Doctor Record

Doctor ID : 201

Doctor Name : Dr. Kumar

Experiment 8: Online Shopping Order System using Packages

Aim

To develop an **Online Shopping Order System** using Java **Packages** by creating separate packages shopping.product and shopping.order to implement product and order management, and calculate the total bill.

Algorithm

1. Start the program.
2. Create the package shopping.product and define the Product class.
3. Create the package shopping.order and define the Order class.
4. Import both packages into the main application.
5. Accept product details from the user.
6. Calculate the total amount.
7. Display the order details and total bill.
8. Stop the program.

Folder Structure

shopping/

product/

Product.java

order/

Order.java

Main.java

Product.java (shopping/product)

package shopping.product;

```
public class Product {
    public int id;
    public String name;
    public double price;

    public Product(int id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }
}
```

Order.java (shopping/order)

```
package shopping.order;

import shopping.product.Product;

public class Order {

    public void placeOrder(Product p, int quantity) {

        double total = p.price * quantity;
```

```
        System.out.println("Product ID : " + p.id);
        System.out.println("Product Name : " + p.name);
        System.out.println("Price : " + p.price);
        System.out.println("Quantity : " + quantity);
        System.out.println("Total Amount : " + total);
    }
}
```

Main.java

```
import java.util.Scanner;
import shopping.product.Product;
import shopping.order.Order;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Product ID: ");
        int id = sc.nextInt();
        sc.nextLine();
```

```
        System.out.print("Enter Product Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Product Price: ");
        double price = sc.nextDouble();

        System.out.print("Enter Quantity: ");
        int qty = sc.nextInt();

        Product p = new Product(id, name, price);
        Order o = new Order();

        o.placeOrder(p, qty);

        sc.close();
    }
}
```

Sample Input

```
101
Laptop
50000
2
```

Sample Output

Enter Product ID: 101

Enter Product Name: Laptop

Enter Product Price: 50000

Enter Quantity: 2

Product ID : 101

Product Name : Laptop

Price : 50000.0

Quantity : 2

Total Amount : 100000.0

Experiment 9: Student Result Analyzer using Arrays of Objects

Aim

To develop a **Student Result Analyzer** using Java **Arrays of Objects** to store student details and calculate **total marks, average, grade, and class topper**.

Algorithm

1. Start the program.
2. Create a Studentclass with data members:
   - Roll Number
   - Name
   - Marks in three subjects
3. Create methods to:
   - Calculate Total
   - Calculate Average
   - Find Grade
   - Display Student Details
4. Create an array of Student objects.
5. Accept details of multiple students.
6. Display each student's result.
7. Compare total marks and identify the class topper.
8. Display the topper details.
9. Stop the program.

Java Code

```
import java.util.Scanner;


class Student {
   int rollNo;
   String name;
   int m1, m2, m3;
```

```
Student(int rollNo, String name, int m1, int m2, int m3) {
    this.rollNo = rollNo;
    this.name = name;
    this.m1 = m1;
    this.m2 = m2;
    this.m3 = m3;
}

int total() {
    return m1 + m2 + m3;
}

double average() {
    return total() / 3.0;
}

String grade() {
    double avg = average();

    if (avg >= 90)
        return "A";
    else if (avg >= 75)
        return "B";
```

```
        else if (avg >= 50)
            return "C";
        else
            return "Fail";
    }

    void display() {
        System.out.println("\nRoll No : " + rollNo);
        System.out.println("Name : " + name);
        System.out.println("Total : " + total());
        System.out.println("Average : " + average());
        System.out.println("Grade : " + grade());
    }
}

public class StudentResultAnalyzer {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Number of Students: ");
        int n = sc.nextInt();
```

```
Student[] s = new Student[n];

for (int i = 0; i < n; i++) {

    System.out.println("\nStudent " + (i + 1));

    System.out.print("Roll No: ");
    int roll = sc.nextInt();
    sc.nextLine();

    System.out.print("Name: ");
    String name = sc.nextLine();

    System.out.print("Mark 1: ");
    int m1 = sc.nextInt();

    System.out.print("Mark 2: ");
    int m2 = sc.nextInt();

    System.out.print("Mark 3: ");
    int m3 = sc.nextInt();

    s[i] = new Student(roll, name, m1, m2, m3);
```

```
        }

        int top = 0;

        for (int i = 0; i < n; i++) {
            s[i].display();

            if (s[i].total() > s[top].total()) {
                top = i;
            }
        }

        System.out.println("\nClass Topper");
        s[top].display();

        sc.close();
    }
}
```

Sample Input

```
Enter Number of Students: 2

Student 1
```

Roll No: 101

Name: Rahul

Mark 1: 90

Mark 2: 95

Mark 3: 92

Student 2

Roll No: 102

Name: Priya

Mark 1: 80

Mark 2: 85

Mark 3: 88

Sample Output

Roll No : 101

Name : Rahul

Total : 277

Average : 92.33

Grade : A

Roll No : 102

Name : Priya

Total : 253

Average : 84.33

Grade : B

Class Topper

Roll No : 101

Name : Rahul

Total : 277

Average : 92.33

Grade : A

Experiment 10: Smart Home Device Controller using Hierarchical Inheritance

Aim

To develop a **Smart Home Device Controller** using Java **Hierarchical Inheritance** by creating a base class Device and derived classes Light, Fan, and AirConditioner. The program uses runtime polymorphism to control devices and display their power consumption.

Algorithm

1. Start the program.
2. Create a base class Devicewith methods:
   - turnOn()
   - turnOff()
   - powerConsumption()
3. Create subclasses:
   - Light
   - Fan
   - AirConditioner
4. Override the methods in each subclass.
5. Store objects in an array of Device references.
6. Display each device status and power consumption using polymorphism.
7. Stop the program.

Java Code

```
class Device {


  void turnOn() {

    System.out.println("Device Turned ON");

  }



  void turnOff() {

    System.out.println("Device Turned OFF");
```

```
    }

    void powerConsumption() {
        System.out.println("Power Consumption");
    }
}

class Light extends Device {

    void turnOn() {
        System.out.println("Light is ON");
    }

    void turnOff() {
        System.out.println("Light is OFF");
    }

    void powerConsumption() {
        System.out.println("Power Consumption : 20 Watts");
    }
}

class Fan extends Device {
```

```
    void turnOn() {
        System.out.println("Fan is ON");
    }

    void turnOff() {
        System.out.println("Fan is OFF");
    }

    void powerConsumption() {
        System.out.println("Power Consumption : 75 Watts");
    }
}

class AirConditioner extends Device {

    void turnOn() {
        System.out.println("Air Conditioner is ON");
    }

    void turnOff() {
        System.out.println("Air Conditioner is OFF");
    }
```

```
    void powerConsumption() {
        System.out.println("Power Consumption : 1500 Watts");
    }
}

public class SmartHome {

    public static void main(String[] args) {

        Device[] devices = new Device[3];

        devices[0] = new Light();
        devices[1] = new Fan();
        devices[2] = new AirConditioner();

        for (Device d : devices) {

            d.turnOn();
            d.powerConsumption();
            d.turnOff();

            System.out.println();
```

```
    }
  }
}
```

Sample Input

Enter Device Choice:

1. **Light**
2. **Fan**
3. **Air Conditioner**

1

Sample Output

Light is ON

Power Consumption : 20 Watts

Light is OFF